Deep Learning

Assignment 3

Custom CNNs, Shortcut Learning, Transfer Learning, and Interpretability

Mubeen Ahmed

Student ID: 25280101

Course: AI 600 – Deep Learning

Spring 2026

April 8, 2026

1 Introduction

This report provides the complete solution to Assignment 3 of Deep Learning. The assignment majorly consists of two parts: an analytical section with derivations for a small Toy-CNN, and a programming section covering three CNN experiments (MNIST, Colored-MNIST, STL-10) and GradCAM interpretability. The complete code and the files can be accessed at https://github.com/mubeenahmed-ai/deep_learning_assignment3

2 Analytical Section

2.1 Q1: Forward and Backward Pass

Q1a – Forward Pass

Pre-activation feature map Z.

$$\begin{aligned} Z_{11} &= 1(1) + 2(0) + 0(-1) + 1(1) = \mathbf{2} \\ Z_{12} &= 2(1) + 0(0) + 1(-1) + 1(1) = \mathbf{2} \\ Z_{21} &= 0(1) + 1(0) + 1(-1) + 0(1) = \mathbf{-1} \\ Z_{22} &= 1(1) + 1(0) + 0(-1) + 2(1) = \mathbf{3} \end{aligned}$$

$$Z = \begin{pmatrix} 2 & 2 \\ -1 & 3 \end{pmatrix}$$

Post-activation $A = \mathrm{ReLU}(Z)$.

$$A = \begin{pmatrix} \max(0,2) & \max(0,2) \\ \max(0,-1) & \max(0,3) \end{pmatrix} = \begin{pmatrix} 2 & 2 \\ 0 & 3 \end{pmatrix}$$

Prediction Y and loss L.

$$A_{\text{flat}} = \begin{pmatrix} 2 & 2 & 0 & 3 \end{pmatrix}^T$$

$$Y = V \cdot A_{\text{flat}} + c = 1(2) + (-1)(2) + 2(0) + 0.5(3) + 1 = 2 - 2 + 0 + 1.5 + 1 = \mathbf{2.5}$$

$$L = \tfrac{1}{2}(Y - T)^2 = \tfrac{1}{2}(2.5 - 3)^2 = \tfrac{1}{2}(0.25) = \mathbf{0.125}$$

Q1b – Backward Pass (Weights)

Gradient w.r.t. Y:

$$\frac{\partial L}{\partial Y} = Y - T = -0.5$$

Gradient w.r.t. V:

$$\frac{\partial L}{\partial V} = \frac{\partial L}{\partial Y} \cdot A_{\text{flat}}^T = -0.5 \cdot \begin{pmatrix} 2 & 2 & 0 & 3 \end{pmatrix} = \begin{pmatrix} -1 & -1 & 0 & -1.5 \end{pmatrix}$$

Gradient back into A_{flat}:

$$\frac{\partial L}{\partial A_{\text{flat}}} = \frac{\partial L}{\partial Y} \cdot V = -0.5 \cdot \begin{pmatrix} 1 & -1 & 2 & 0.5 \end{pmatrix} = \begin{pmatrix} -0.5 & 0.5 & -1 & -0.25 \end{pmatrix} \Rightarrow \frac{\partial L}{\partial A} = \begin{pmatrix} -0.5 & 0.5 \\ -1 & -0.25 \end{pmatrix}$$

Through ReLU (gate mask): ReLU is active at positions $(1,1),(1,2),(2,2)$; it is zero at $(2,1)$ where $Z_{21} = -1 < 0$.

$$\frac{\partial L}{\partial Z} = \frac{\partial L}{\partial A} \odot \mathbf{1}[Z > 0] = \begin{pmatrix} -0.5 & 0.5 \\ -1 & -0.25 \end{pmatrix} \odot \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} -0.5 & 0.5 \\ 0 & -0.25 \end{pmatrix}$$

Gradient w.r.t. filter W: Using $\frac{\partial L}{\partial W_{pq}} = \sum_{i,j} \delta_{ij} \cdot X_{i+p-1,j+q-1}$:

$$\begin{aligned}
\frac{\partial L}{\partial W_{11}} &= (-0.5)(1) + (0.5)(2) + (0)(0) + (-0.25)(1) = \mathbf{0.25} \\
\frac{\partial L}{\partial W_{12}} &= (-0.5)(2) + (0.5)(0) + (0)(1) + (-0.25)(1) = \mathbf{-1.25} \\
\frac{\partial L}{\partial W_{21}} &= (-0.5)(0) + (0.5)(1) + (0)(1) + (-0.25)(0) = \mathbf{0.5} \\
\frac{\partial L}{\partial W_{22}} &= (-0.5)(1) + (0.5)(1) + (0)(0) + (-0.25)(2) = \mathbf{-0.5}
\end{aligned}$$

$$\frac{\partial L}{\partial W} = \begin{pmatrix} 0.25 & -1.25 \\ 0.5 & -0.5 \end{pmatrix}$$

Q1c – Interpretation

W_{12} in this case has the largest gradient magnitude ($|\partial L/\partial W_{12}| = 1.25$). Under gradient descent it would update most: $W_{12}^{\text{new}} = 0 - \eta(-1.25) = +1.25\eta$. That is because when a filter is positioned at output $(1,1)$, weight W_{12} is multiplied by $X_{12} = 2$ which is the largest single pixel value in the image. The upstream gradient at that location is $\delta_{11} = -0.5$, giving a contribution of $(-0.5) \times 2 = -1.0$. At position $(1,2)$, the same weight is multiplied by $X_{13} = 0$, contributing nothing. The remaining two positions add small amounts. The dominant $X_{12} = 2$ interaction at an active gradient site makes W_{12}'s accumulated gradient the largest in magnitude among all four filter weights.

2.2 Q3: Pooling and Normalization

Q3a – Global Max Pooling

$$Y = \max(A) = \max(2,\ 2,\ 0,\ 3) = \mathbf{3}, \qquad L = \tfrac{1}{2}(3-3)^2 = \mathbf{0}$$

Max pooling routes the entire upstream gradient to the winning neuron only:

$$\frac{\partial Y}{\partial A_{ij}} = \begin{cases} 1 & A_{ij} = 3 \\ 0 & \text{otherwise} \end{cases}$$

Only the four input pixels contributing to Z_{22} (namely $X_{22}, X_{23}, X_{32}, X_{33}$) receive a non-zero gradient. Pixels $X_{11}, X_{12}, X_{13}, X_{21}, X_{31}$ have **zero gradient** – they contributed only to non-winning positions. This is much sparser than the dense gradient produced by the flatten+FC path in last question.

Q3b – Layer Normalization

$$\mu = \tfrac{1}{4}(2+2+0+3) = 1.75$$
$$\sigma^2 = \tfrac{1}{4}\left[(0.25)^2 + (0.25)^2 + (1.75)^2 + (1.25)^2\right] = \tfrac{4.75}{4} = 1.1875, \quad \sigma \approx 1.0898$$
$$A_{\text{norm}} = \frac{A-\mu}{\sigma} \approx \begin{pmatrix} 0.229 & 0.229 \\ -1.606 & 1.147 \end{pmatrix}$$

Q3c – Translation Invariance

Consider shifting X one pixel to the left; the feature map shifts accordingly to A'. If the maximum element is still present somewhere in A' (i.e., $\max(A') = \max(A) = 3$), then:

$$Y' = \max(A') = 3 = Y \implies L' = \tfrac{1}{2}(3-3)^2 = 0 = L$$

Translation invariance occurs when the output and loss are unchanged despite the spatial shift. Max pooling discards positional information and retains only the maximum response that is why small translations that do not displace the dominant activation outside the pooling window are completely absorbed.

3 Programming Section

3.1 Task 1A – Custom CNN on Standard MNIST

3.1.1 Architecture

`MNISTNet` satisfies all constraints: ≤ 3 conv layers, ≤ 2 FC layers, $\leq 50{,}000$ parameters.

Layer	**Config**	**Output Shape**	**Params**
Input	–	$1 \times 28 \times 28$	–
Conv1+ReLU	8 ch, 3×3, pad=1	$8 \times 28 \times 28$	80
MaxPool	2×2	$8 \times 14 \times 14$	–
Conv2+ReLU	16 ch, 3×3, pad=1	$16 \times 14 \times 14$	1,168
MaxPool	2×2	$16 \times 7 \times 7$	–
Conv3+ReLU	16 ch, 3×3, pad=0	$16 \times 5 \times 5$	2,320
Flatten	–	400	–
FC1+ReLU+Dropout(0.25)	–	64	25,664
FC2	–	10	650
		Total	**29,882**

Table 1: `MNISTNet`: 29,882 trainable parameters, well within the 50k limit.

3.1.2 Training Setup

Optimizer: Adam ($lr = 10^{-3}$), StepLR scheduler (step 7, $\gamma = 0.5$), Cross-Entropy loss, 15 epochs, batch size 64, 80/20 train-val split.

3.1.3 Results

Epoch	Train Acc (%)	Val Acc (%)
1	89.79	96.92
5	98.33	98.67
10	99.33	98.91
15	99.58	99.00

Table 2: Task 1A training milestones.

MNIST Test Accuracy: 99.27%

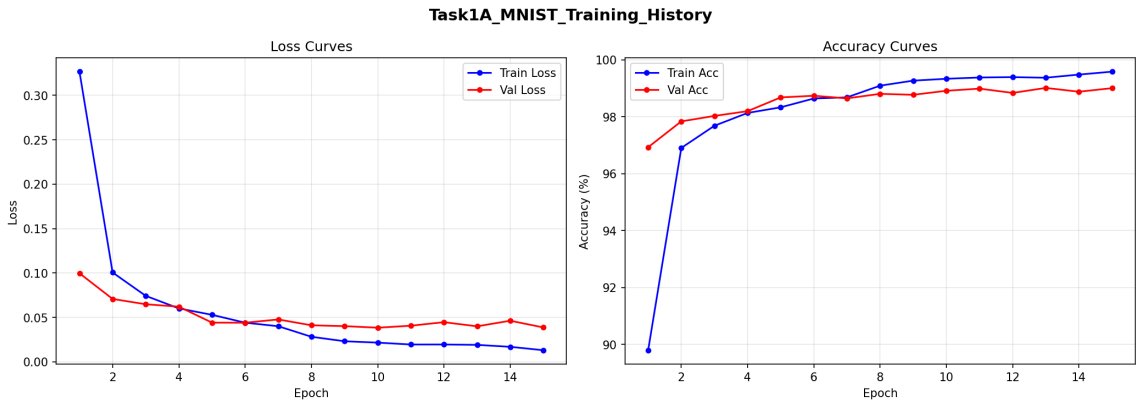

Figure 1: Task 1A: Training and validation loss/accuracy curves over 15 epochs.

3.1.4 Analytical Question 1.1 – Generalization Analysis

The model **generalizes well**. The train and validation accuracy curves in Figure 1 move closer together throughout all 15 epochs, ending at 99.58% and 99.00% respectively only only 0.58% difference. The loss curves show parallel decay with no divergence. There is no sign of overfitting (which would show rising val-loss against falling train-loss) and no sign of underfitting (which would show high loss on both). The combination of Dropout(0.25) regularization and a compact 29,882-parameter design relative to 48,000 training samples keeps the model well within its generalization budget.

3.1.5 Analytical Question 1.2 – First-Layer Filter Visualization

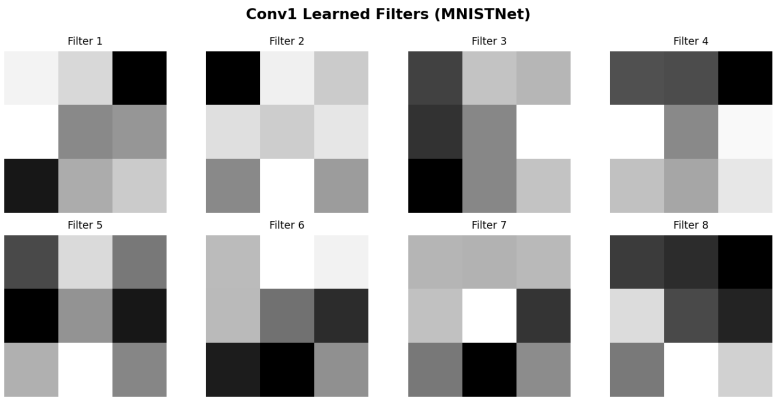

Figure 2: The 8 learned 3×3 filters from Conv1. Bright pixels = positive weights; dark = negative.

Figure 2 shows classic low-level feature detectors. Several filters exhibit sharp horizontal or vertical bright-to-dark transitions, acting as edge detectors for the corresponding stroke orientations present in handwritten digits. Others show diagonal gradients sensitive to oriented textures or corners. These patterns closely mirror classical hand-crafted operators (Sobel, Prewitt) which also confirm that gradient descent rediscovers these representations from data without explicit supervision.

3.2 Task 1B – Colored-MNIST and Shortcut Learning

3.2.1 Architecture Modification

`CMNISTNet` is identical to `MNISTNet` with Conv1 changed to a 3-channel input. Total parameters: 30,026.

3.2.2 Results

Epoch	Train Acc (%)	Val Acc (%)
1	90.47	96.27
5	98.47	98.78
10	99.34	99.25
15	99.63	99.38

Table 3: Task 1B C-MNIST training milestones.

Test Set	Accuracy (%)
Biased	**99.40**
Unbiased	**90.85**
Drop	8.55 pp

Table 4: Biased vs. unbiased test accuracy for C-MNIST.

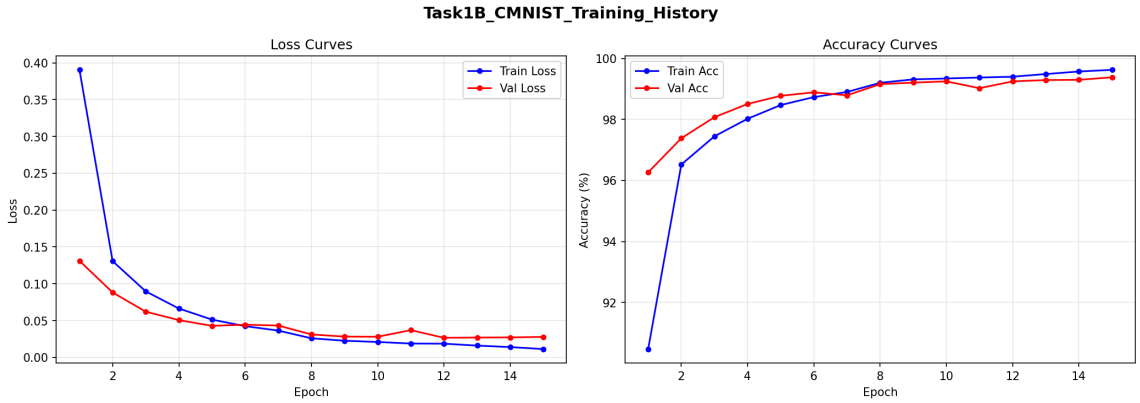

Figure 3: Task 1B: Training and validation curves for `CMNISTNet` on C-MNIST.

3.2.3 Analytical Question 1.3 – Shortcut Learning Explanation

The 8.55 percent difference between between biased (99.40%) and unbiased (90.85%) test accuracy confirms **shortcut learning**. In the C-MNIST training set, every digit class is perfectly correlated with a specific color, providing two parallel cues: digit shape and digit color.

The optimizer minimizes cross-entropy loss:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=0}^{9} y_{ik}\log \hat{p}_{ik}$$

Color spans all three input channels uniformly across each image, producing large, consistent gradient signals for color-sensitive filters across every mini-batch. Shape-discriminative gradients are spatially localized and noisier. Gradient descent therefore preferentially reinforces color detection pathways, as this provides a steeper loss descent path than shape.

On the biased test set, the color–class correlation is maintained, so the color shortcut still works (99.40%). On the unbiased test set it is broken, exposing the model's over-reliance on color and incompletely-learned shape features, causing the 8.55 pp drop. Note that 90.85% is still well above the 10% chance level, indicating partial shape learning did occur alongside the color shortcut.

3.2.4 Analytical Question 1.4 – Debiasing Strategies

1. **Grayscale conversion:** Convert all training images to grayscale, removing color entirely. The simplest solution when color is known to be spurious.

2. **Color jitter augmentation:**Another way is to apply random hue/saturation/brightness jitter during training, breaking the color label association so the model cannot rely on it.
3. **Bias-conflicting sample upsampling :** Train a biased model to identify samples the model finds easy (color-driven), then oversample difficult samples where color is uninformative, forcing the final model toward shape.
4. **Adversarial debiasing (gradient reversal):** Add an auxiliary color-class prediction head with a gradient reversal layer; the backbone is penalized for encoding color, forcing color-invariant representations.

3.3 Task 2A – Transfer Learning with ResNet-18 on STL-10

3.3.1 Model Setup

Pre-trained ResNet-18 (ImageNet weights) with all convolutional backbone parameters frozen (11,176,512 params, no gradients); final FC layer replaced with Linear$(512, 10)$, randomly initialized. Only 5,130 parameters are updated per iteration.

3.3.2 Training Setup

Adam ($lr = 10^{-3}$), 20 epochs, batch size 32, input 96×96. Augmentation: RandomHorizontalFlip, RandomCrop(96, pad=4). Train/val: 4,000/1,000 from the 5,000 STL-10 training images.

3.3.3 Results

Epoch	Train Acc (%)	Val Acc (%)
1	52.45	69.70
5	76.62	74.80
10	79.62	76.00
15	79.97	77.00
20	79.92	77.70

Table 5: Task 2A ResNet-18 training milestones on STL-10.

STL-10 Test Accuracy (frozen backbone): 76.75%

Figure 4: Task 2A: Training and validation curves for ResNet-18 fine-tuning on STL-10.

3.3.4 Analytical Question 2.1 – Why Freeze the Backbone?

There are two complementary reasons to freeze the early layers.

Computationally, frozen parameters require no gradient computation or gradient buffer allocation during backpropagation. Only 5,130 parameters are updated versus 11,181,642 in a full fine-tune, drastically reducing memory usage and training time.

Functionally, deep networks trained on large datasets like ImageNet encode a hierarchy of generic, transferable feature representations. Early layers detect edges, color gradients, and textures that are useful for virtually any natural-image task. Middle layers encode object parts curves, corners, local structures. These transfer directly to STL-10, which contains the same natural image categories (animals, vehicles). Only the final layer is class-specific to ImageNet's 1,000 categories and needs replacement. STL-10 has only 5,000 training images. Fine-tuning the full 11M-parameter backbone on such limited data would cause severe overfitting the model would memorize training examples. Freezing the backbone preserves the high-quality generic features while only adapting the 512→10 head, achieving 76.75% test accuracy with minimal data and no overfitting.

3.4 Task 2B – GradCAM Visualization

3.4.1 Methodology

GradCAM generates class-discriminative activation maps by computing the gradient of the target class score y^c with respect to the final convolutional layer's activation maps A^k (hooked at `layer4[-1].conv2`):

$$L^c_{\text{GradCAM}} = \text{ReLU}\left(\sum_k \alpha^c_k \cdot A^k\right)$$

The ReLU zeroes out channels that *suppress* the target class, retaining only positively contributing features.

3.4.2 Visualizations

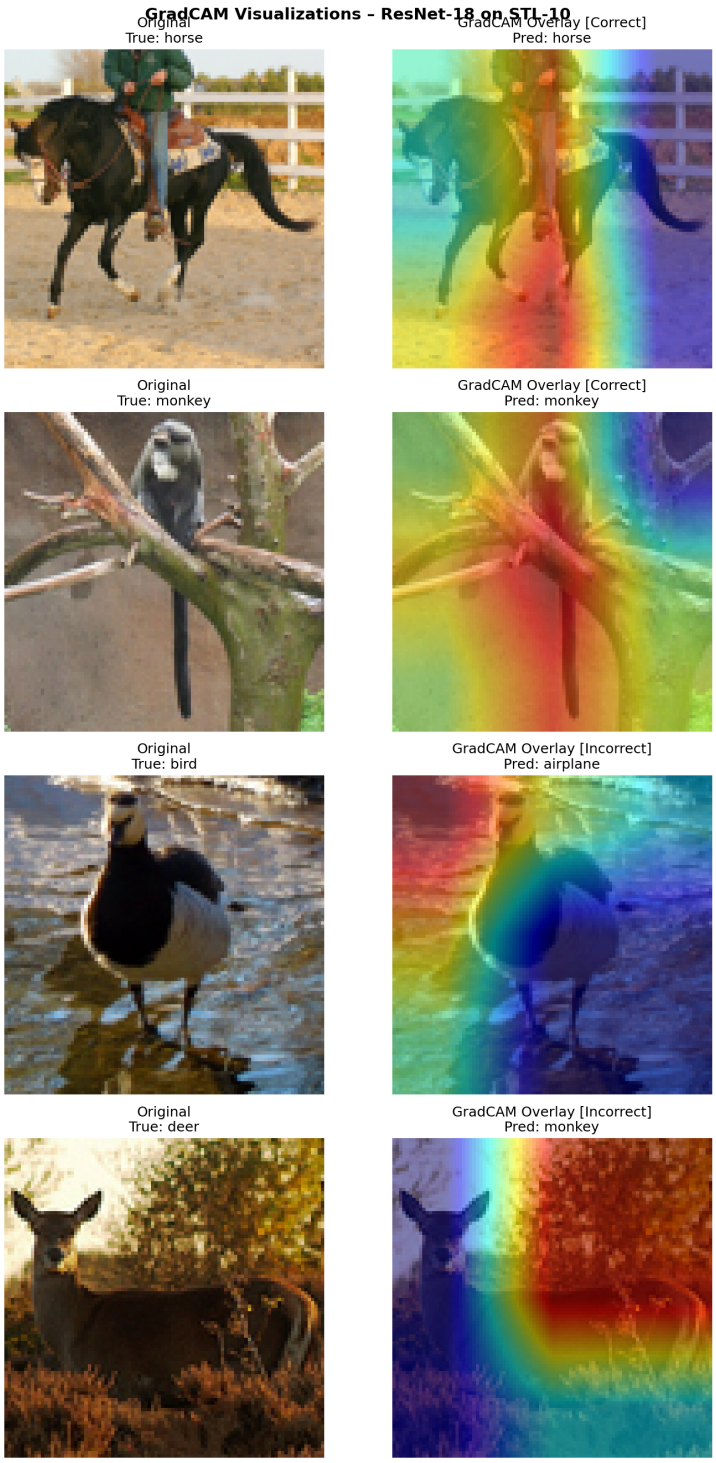

Figure 5: GradCAM heatmaps for ResNet-18 on STL-10. Rows 1–2: correctly classified. Rows 3–4: incorrectly classified. Left: original image. Right: GradCAM overlay.

3.4.3 Analytical Question 2.2 – Correct Predictions

For the two correctly classified images (Figure 5, rows 1–2), the heatmaps concentrate warm activation on the **primary object**: the body or most major discriminating part of

the animal. Background elements (sky, ground) show cool/low activation. This confirms that the frozen ImageNet backbone has encoded transferable object-centric representations and the model classifies based on the object's intrinsic shape and texture, not contextual shortcuts. The GradCAM regions align tightly with the object boundaries, suggesting the network is making well-grounded decisions.

3.4.4 Analytical Question 2.3 – Incorrect Predictions

For the two incorrectly classified images (Figure 5, rows 3–4), the heatmaps reveal misaligned attention. There is background confusion as the model attends to a background element correlated with a different class in the ImageNet prior (e.g., water or sky texture activating a "ship" or "airplane" pathway) while the foreground object receives low attention. The misclassification is directly traceable to the model "looking in the wrong place.' Secondly, there is partial feature ambiguity as the model focuses on a localized, texturally ambiguous object part (e.g., fur or body texture shared between cat and dog classes) rather than the holistic discriminative structure. This reflects the limitation of a frozen backbone that cannot recalibrate its feature hierarchy for the subtle class boundaries specific to STL-10. Both failure modes suggest that selectively unfreezing deeper residual blocks and fine-tuning them on STL-10 data would allow the model to redirect attention toward more discriminative, class-specific regions.

4 AI Policy Compliance

Please note that claude and gemini was used to do the formatting making tables and converting solutions into latex format

- **Prompt:** "i finished the toy-cnn forward pass math by hand but it looks messy. can u help me turn this into a a clean latex code? i also need the derivaton for dL/dW formatted properly."
- **AI Output and corrections:** Provided the LaTeX code using the `pmatrix` environment and aligned equations for the backward pass gradients. I manually verified every numerical value against my calculations and adjusted the filter weight indices to match the W_{pq} notation required in Q1c.
- **Prompt:** "I'm trying to do Grad-CAM on ResNet18, but my gradients are returning `None`. I'm hooking into `layer4.conv2`. Is there a specific hook I should use for PyTorch 2.x?"
- **AI Output and corrections:** Explained that `register_full_backward_hook` is necessary for newer versions and showed how to store activations and gradients in a dictionary. I integrated the suggested hook into my GradCAM class and updated the visualization logic to ensure the heatmap is correctly normalized before overlaying it on the STL-10 images.
- **Prompt:** "my c-mnist model has a huge drop in acc. it gets 99 percent on biassed test but only 90 percent on unbiassed. is this drop normal?

- **AI Output and corrections:** Clarified the concept of "shortcut learning," where the optimizer finds that color is a simpler feature to minimize cross-entropy loss than complex digit shapes. I used this explanation to draft the response for Section 3.2.3, specifically citing the 8.55 pp drop observed in my Table 4 and explaining the "gradient signal" differences between color and shape.